SQL Introduction (Detailed Guide for PDF)

What is SQL?

SQL (Structured Query Language) is a standard language used to manage and manipulate relational databases.

It is used to:

- Store data
- Retrieve data
- Update data
- Delete data

History of SQL

- Developed in 1970s at IBM
- Originally called SEQUEL
- Standardized by ANSI and ISO

Why Use SQL?

- Easy to learn
- Powerful for data handling
- Used in almost all applications
- Supports large databases

Types of SQL Commands

1. DDL (Data Definition Language)

- CREATE
- ALTER
- DROP

2. DML (Data Manipulation Language)

- INSERT
- UPDATE
- DELETE

3. DQL (Data Query Language)

- SELECT

4. DCL (Data Control Language)

- GRANT
- REVOKE

SQL Syntax Example

```
SELECT * FROM students;
```

Creating a Table

```
CREATE TABLE students (
  id INT,
  name VARCHAR(50),
  age INT
);
```

Inserting Data

```
INSERT INTO students VALUES (1, 'John', 20);
```

Selecting Data

```
SELECT name, age FROM students;
```

WHERE Clause

```
SELECT * FROM students WHERE age > 18;
```

Updating Data

```
UPDATE students SET age = 21 WHERE id = 1;
```

Deleting Data

```
DELETE FROM students WHERE id = 1;
```

SQL Operators

- =, >, <
- AND, OR, NOT
- LIKE

SQL Constraints

- NOT NULL
- UNIQUE
- PRIMARY KEY
- FOREIGN KEY

Example:

```
id INT PRIMARY KEY
```

Joins in SQL

INNER JOIN

```
SELECT * FROM A INNER JOIN B ON A.id = B.id;
```

LEFT JOIN

```
SELECT * FROM A LEFT JOIN B ON A.id = B.id;
```

Aggregate Functions

- COUNT()
- SUM()
- AVG()

- MAX()
- MIN()

Example:

```
SELECT COUNT(*) FROM students;
```

GROUP BY

```
SELECT age, COUNT(*) FROM students GROUP BY age;
```

ORDER BY

```
SELECT * FROM students ORDER BY age DESC;
```

Indexes

Used to improve query performance.

```
CREATE INDEX idx_name ON students(name);
```

Views

```
CREATE VIEW student_view AS
SELECT name FROM students;
```

Normalization

Process of organizing data to reduce redundancy.

Example Project

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  salary INT
);

INSERT INTO employees VALUES (1, 'Alice', 50000);
```

```
SELECT * FROM employees;
```

Advantages of SQL

- Easy to use
- Efficient data handling
- Standard language

Disadvantages of SQL

- Complex queries can be difficult
- Limited control compared to programming languages

Best Practices

- Use proper naming conventions
- Normalize data
- Use indexes wisely
- Backup data regularly